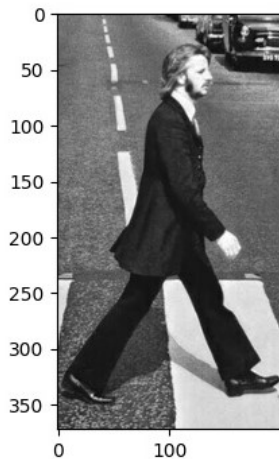


DÉTECTION DES PIÉTONS DANS UNE IMAGE

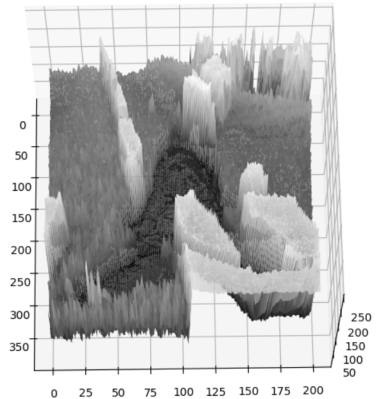
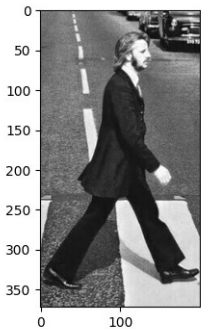
REGRAGUI ILYASS

CPGE IBN BAJA , BENI MELLAL

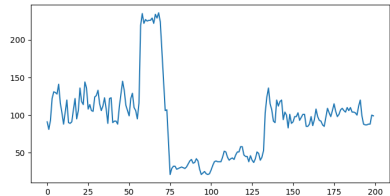
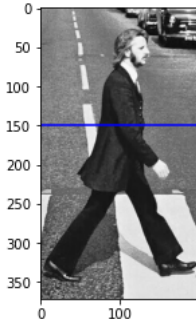
July 17, 2023



L'image est une fonction : $I : (x, y) \longrightarrow I(x, y)$
Qui associe à chaque couple (x, y) un niveau de gris.



Le contour est une variation brusque d'intensité. (Donc, pour le détecter, on recourt au calcul de dérivé d'image. (gradient))



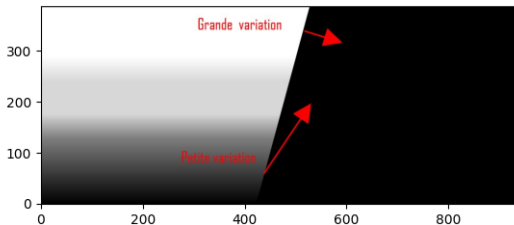
vecteur gradient

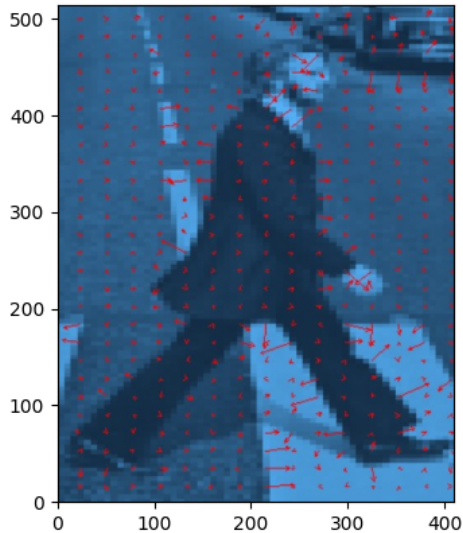
Possède :

Une norme $n = ((\frac{\delta I(x,y)}{\delta x})^2 + (\frac{\delta I(x,y)}{\delta y})^2)^{1/2}$

Une direction : $\theta = \arctan(\frac{\delta I(x,y)}{\delta y} / \frac{\delta I(x,y)}{\delta x})$

Perpendiculaire au contour .





Problème

Considérer tous les vecteurs gradients d'une image pour la détection d'un objet peut entraîner un temps de calcul trop élevé, ce qui n'est pas souhaitable pour l'optimisation des calculs.

Nécessité de regrouper les vecteurs gradients en plusieurs intervalles d'orientation et de calculer un histogramme pour chaque groupe.

Vecteur de caractéristiques

Discrétisation de l'orientation

La discrétisation de l'orientation est donnée par la formule :

$$\beta(x, y) = \text{round} \left(\frac{p \cdot \theta(x, y)}{2\pi} \right) [2\pi]$$

Avec

- (x, y) sont les coordonnées du pixel
- p est le nombre de classes (groupes)
- $\theta(x, y)$ est l'orientation du gradient.

Vecteur de caractéristiques

On définit ainsi le vecteur de caractéristiques pour une certaine valeur d'orientation b :

$$F(x, y)_b = \begin{cases} r(x, y) & \text{si } b = \beta(x, y) \\ 0 & \text{sinon} \end{cases}$$

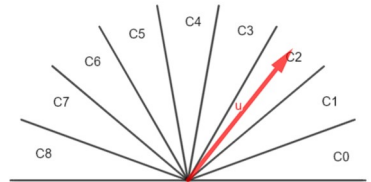
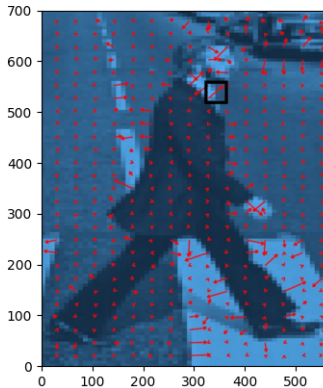
Si on choisit, par exemple, un nombre de classes $p = 9$, les orientations seront réparties de manière équi-angulaire sur une plage de 2π radians (ou 360 degrés).

Cela signifie que les orientations du gradient sont regroupées en 9 classes différentes, chacune représentant un intervalle d'orientation spécifique.

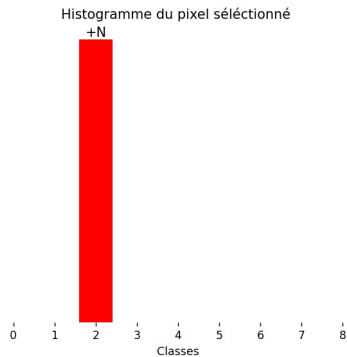
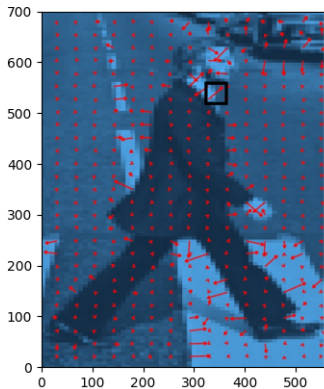
La formule donnée pour la discrétisation de l'orientation, $\beta(x, y)$, permet de déterminer à quelle classe appartient l'orientation du gradient en un point spécifique (x, y) .

Pour le vecteur sélectionné : il s'agit d'un vecteur d'orientation
 $\theta(x, y) = 70^\circ (\frac{7\pi}{18} rad)$, donc $\beta(x, y) = 2$ et le vecteur de caractéristiques
 est donc:

$$F(x, y)_b = \begin{cases} r(x, y) & \text{si } b = 2 \\ 0 & \text{sinon} \end{cases}$$

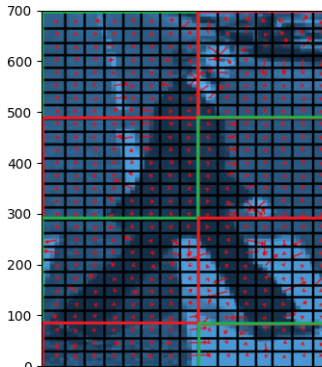


Caractéristique (**histogramme**) du pixel sélectionnés :

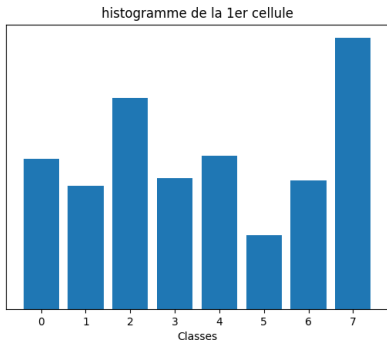


HISTOGRAMME D'ORIENTATION DE GRADIENT

Pour réduire la taille des descripteurs et diminuer la complexité de l'algorithme, il est nécessaire de mapper chaque pixel (x,y) de l'image dans une cellule spécifique définie par des coordonnées $([E(x/k), E(y/k)])$ avec $k \in \mathbb{N} / \{0\}$. Ainsi le vecteur caractéristique de la cellule et la somme des histogramme des pixel .
c'est un sous vecteur **(de dimension 8)** .

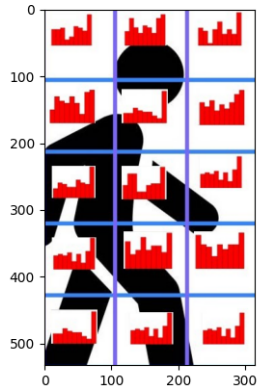
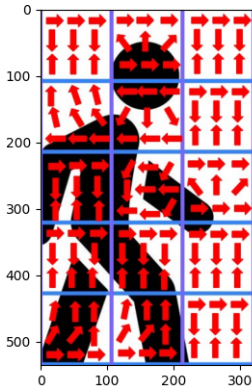


REGRAGUI ILYASS



DÉTECTION DES PIÉTONS DANS UNE IMAGE





d'ou le vecteur caractéristique

:

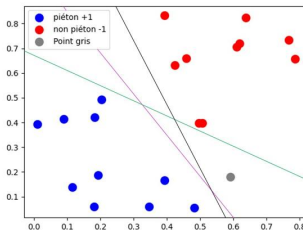
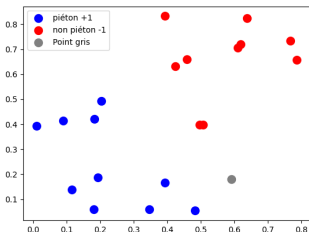


Problème

Maintenant que nous avons extrait avec succès les descripteurs HOG de nos images, comment pouvons-nous utiliser ces descripteurs pour classer efficacement les piétons et les non-piétons ?

On suppose que nous disposons d'un ensemble de données contenant de nombreuses images de piétons et de non-piétons. Chaque image est représentée par un vecteur descripteur qui appartient à un espace de dimension N . Notre objectif est de classer ces images en piétons et non-piétons.

Pour simplifier, supposons que les vecteurs descripteurs de chaque image sont de dimension 2. Le but est de trouver une frontière optimale (ou un hyperplan h avec $\dim(h) = \dim(\text{vecteur descripteurs}) - 1$) pour séparer de manière optimale les deux classes et ainsi déterminer l'appartenance du point gris; **Cette frontière optimale permet de réduire les probabilités de fausse détection et d'améliorer la précision de la classification.**



fonction discriminante

la fonction discriminante linéaire $f : \chi \rightarrow \mathbb{R}$ tel que :

$$f(x) = w^T x + w_0 = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + w_0$$

avec :

χ : ensemble des vecteurs caractéristiques des images

$w = (w_1, w_2, \dots, w_n)^T$: vecteur poids

$$\begin{cases} f(x) > 0 & : x \in \text{pietons} \\ f(x) < 0 & : x \in \text{nonpieton} \end{cases}$$

la frontière $f(x)=0$ est un hyperplan .

le but de svm est de trouver la fonction f optimale à partir d'un ensemble d'apprentissage : $\{(v_1, l_1), (v_2, l_2), \dots, (v_n, l_n)\} \in \mathbb{R}^N \times \{-1, 1\}$

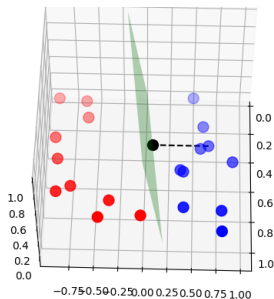
marge géométrique

distance d'un vecteur x au plan Δ :

$$d(x, \Delta) = \frac{|w^T x + w_0|}{\|w\|}$$

la marge est donc :

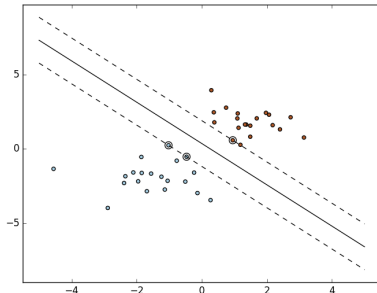
$$\text{marge} = \min_{i \in [1, N]} d(x_i, \Delta(w, w_0))$$



L'algorithme SVM vise à trouver la frontière optimale qui maximise la marge, assurant ainsi une séparation précise des classes et une faible probabilité d'erreur. D'où le problème à résoudre :

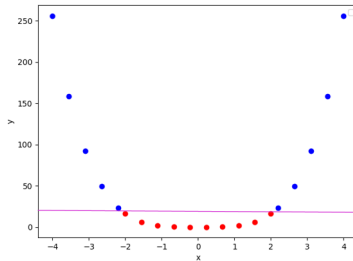
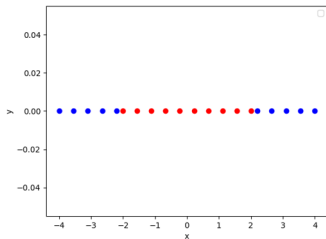
$$\max_{w, w_0} (\min_{i \in [1, N]} \text{dist}(x_i, \Delta(w, w_0)))$$

problème d'optimisation sous contraintes : $\begin{cases} \max_{w, w_0} (\text{marge}) \\ \min_{i \in [1, p]} \left(\frac{|w^T x_i + w_0|}{\|w\|} \right) \geq m \end{cases}$
 que l'on peut résoudre avec des méthode d'optimisations . Comme la méthode des multiplicateurs lagrange .



Cas non linéaire

Avant d'appliquer la fonction x^4 , les vecteurs ne sont pas linéairement séparables.



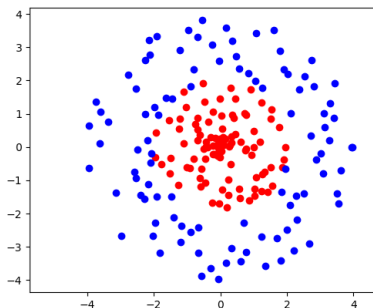
SVM non linéair

L'idée principale des SVM non linéaires est de projeter les données d'entrée de l'espace de description dans un espace de dimension supérieure appelé **redescription**, où il est plus probable qu'une frontière linéaire de séparation existe.

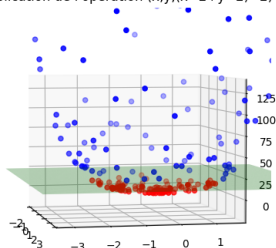
L'opération de redescription est définie par la fonction :

$$\begin{aligned} \Phi : \Omega &\rightarrow \Omega' \\ x &\mapsto \Phi(x) \end{aligned} \quad \text{Appliquons donc un SVM linéaire à}$$

$\{(\Phi(x_i), l_i), i = 1, \dots, N\}$, avec $l_i \in \{-1, 1\}$.



application de l'opération $(x, y, (x^2 + y^2)^2)$



Lorsque nous nous projetons dans un espace de dimension supérieure, les calculs deviennent plus complexes et plus longs. Or, lors de la résolution du problème d'optimisation mentionné précédemment, les seules apparitions de Φ sont de la forme $\Phi(x_i)^T \Phi(x_j)$. Il n'est donc pas nécessaire de connaître Ω' ni Φ , il suffit de connaître toutes les valeurs $\Phi(x_i)^T \Phi(x_j)$ qui ne dépendent que des x_j .

D'où l'introduction de la fonction noyau ou kernel $k(x, x') : \Omega \times \Omega \longrightarrow \mathbb{R}$ définie par $k(x, x') = \Phi(x)^T \Phi(x')$ (**pour diminuer la complexité du calcul de $O(n^2)$ à $O(n)$**). Le kernel doit vérifier la symétrie et la semi-positivité.

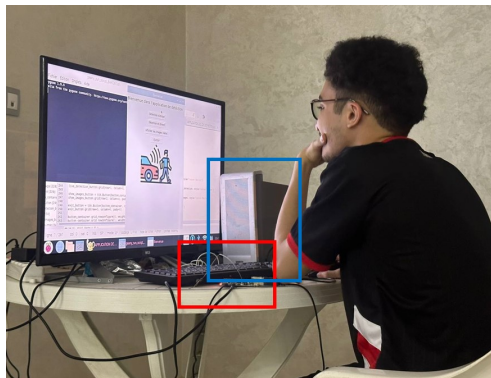
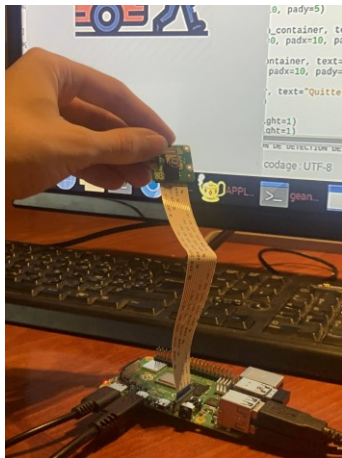
Exemple:

$$K(x, x') = e^{-\frac{|x-x'|^2}{2\sigma^2}}$$

où σ est un paramètre de lissage qui contrôle la largeur de la gaussienne et influence la souplesse de la frontière de décision.

Application de détection des piétons

- . Conçue spécifiquement pour être utilisée sur un Raspberry Pi 4 Model B équipé d'une caméra modul et d'un haut-parleur.
- . Permet la détection précise des piétons dans des images statiques ou en temps réel.
- . Utilise les algorithmes de détection HOG (Histogram of Oriented Gradients) et SVM (Support Vector Machines)
- . Comprend des fonctionnalités d'avertissement pour alerter l'utilisateur en cas de détection de piétons.
- . Facilite la création et la gestion d'une base de données pour stocker et analyser les données collectées.
- . Permet de mesurer les performances du système de détection et de générer des graphiques pour une visualisation claire des résultats obtenus.



Bienvenue dans l'application de détection

Détection statique

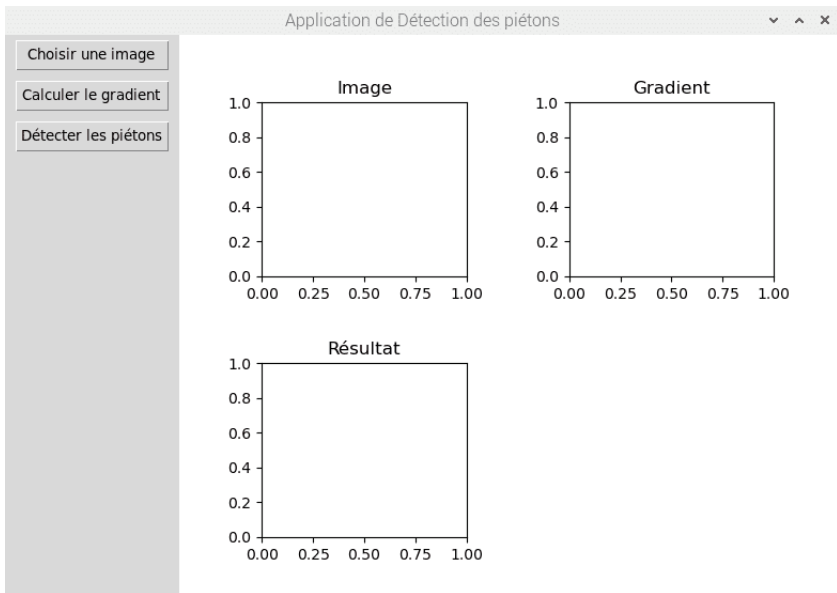
Détection en direct

Afficher les images (data)

Quitter



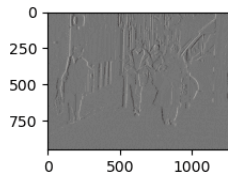
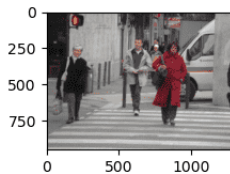
<1er bouton>:



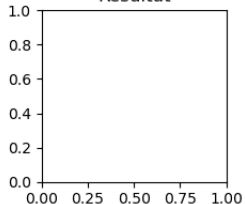
Choisir une image

Calculer le gradient

Détection des piétons



Résultat

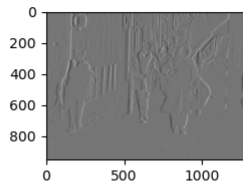


Application de Détection des piétons

Choisir une image

Calculer le gradient

Détecter les piétons



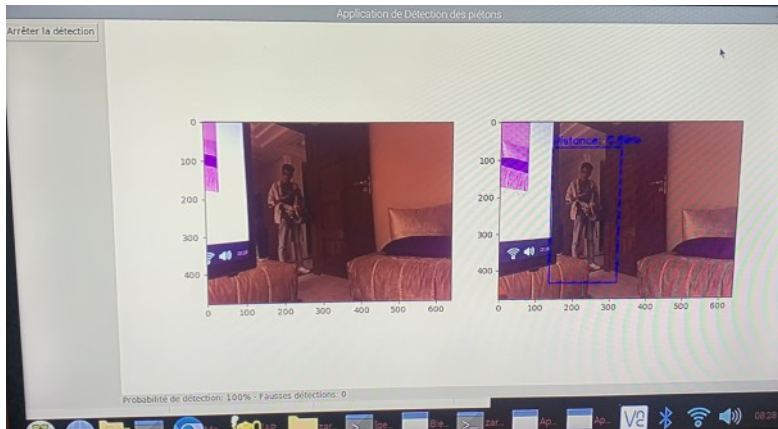
Fichier

<2eme bouton>: lors de détection des piéton, l'application donne une estimation de la distance entre la camera et le piéton, une probabilité de fausse detection, enregistre l'image comme data pour analyser après les performance, avertie le conducteurs à travers le haut parleurs en utilisant les deux bibliotheques sur (**python**): PYTTX3 et PYGAME selon le code (voir tout le code en annexe):

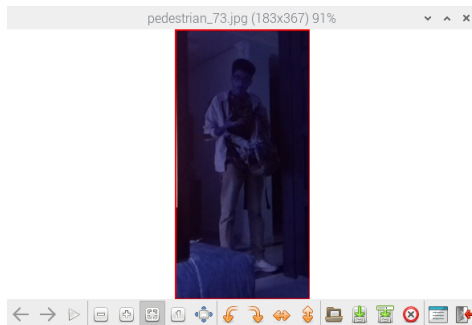
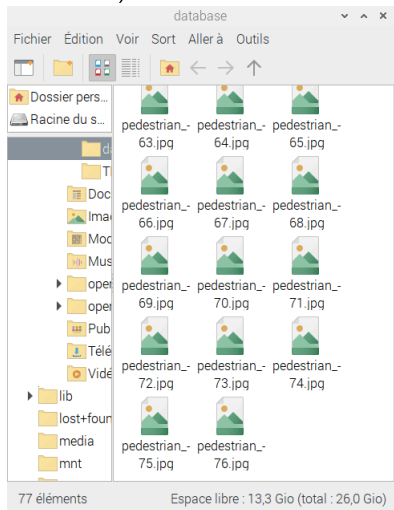
```
# Configuration de l'alarme sonore
pygame.mixer.init()
alarm_sound = pygame.mixer.Sound("/home/zaraki/Téléchargements/Fire_pager-jason-1283464858.wav")
```

et

```
# Alarme sonore et synthèse vocale en fonction de la distance
if distance < 2.0:
    alarm_sound.play()
    engine.say("Danger! Un piéton proche!")
    engine.runAndWait()
elif distance < 5.0:
    alarm_sound.play()
    engine.say("Attention! Un piéton à proximité!")
    engine.runAndWait()
```



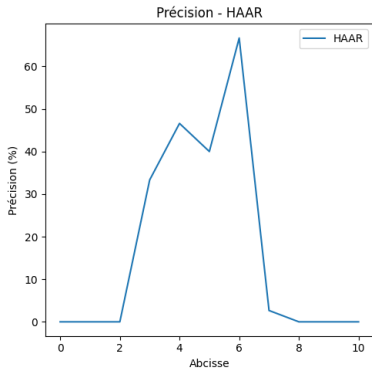
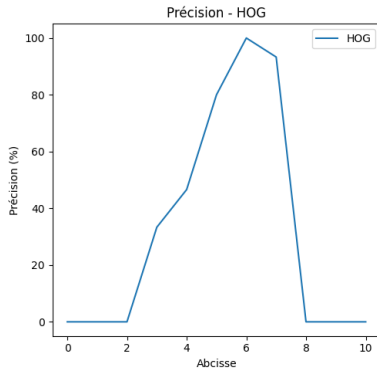
on voit ici une base de donnée riche (qui sera affiché si on clique sur le bouton 3):



Nous avons évalué l'exactitude de notre modèle de détection de piétons en utilisant la base de données INRIA . Nous avons imprimé en couleur 15 images de piétons et pris des photos à différentes distances pour observer la performance du modèle. Cela nous a permis de comprendre l'impact de la distance sur l'exactitude.

Nous avons également étudié la précision de l'application en testant différents descripteurs (HOG) et HAAR CASCADE .

La caméra du module Raspberry Pi est positionnée à une hauteur d'environ 150 cm par rapport au sol, et nous avons varié la distance à l'image de 0 à 10 mètres.



ANNEXE

<importation>:

```
1  #interface graphique
2  import tkinter as tk
3  from tkinter import ttk
4  from tkinter import filedialog
5  #traitement d'image
6  import cv2
7  from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
8  from matplotlib.figure import Figure
9  # effectuer des opérations liées aux fichiers, aux répertoires
10 import os
11 import glob
12 #alarme sonore et sythèse vocale
13 import pyttsx3
14 import pygame
15
```

<fonction bouton détection statique>:

```
23 ~ def start_main_window():
24     main_window = tk.Toplevel(root)
25     main_window.title("Application de Détection des piétons")
26
27 ~ def choisir_image():
28     file_path = filedialog.askopenfilename()
29     img = cv2.imread(file_path, cv2.IMREAD_COLOR)
30     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
31     display_image(img, 1)
32     status.set("Image chargée")
33
34 ~ def calculer_gradient():
35     img = axes[0].images[0].get_array()
36     gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
37     gradient = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=5)
38     display_image(gradient, 2)
```

<fonction bouton détection statique>:

```
42 ~ def detection_piétons():
43     img = axes[0].images[0].get_array()
44     hog = cv2.HOGDescriptor()
45     hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
46     rects, weights = hog.detectMultiScale(img, winStride=(8, 8))
47
48     false_detections = 0
49 ~     if len(rects) > 0:
50 ~         for i, (x, y, w, h) in enumerate(rects):
51 ~             if weights[i] < 0:
52                 false_detections += 1
53                 cv2.rectangle(img, (x, y), (x + w, y + h), (0, 0, 255), 2)
54
55                 detection_percentage = int((len(rects) - false_detections) / len(rects) * 100)
56                 result_text.set(f"Probabilité de détection: {detection_percentage}% - Fausses détection
57 ~     else:
58         result_text.set("Aucune détection")
59
60     display_image(img, 3)
```

<fonction bouton détection statique>:

```
62  def display_image(img, axis):
63      axes[axis - 1].clear()
64      axes[axis - 1].imshow(img, cmap='gray' if len(img.shape) == 2 else None)
65      figure_canvas.draw()
66
67  buttons = [
68      ("Choisir une image", choisir_image),
69      ("Calculer le gradient", calculer_gradient),
70      ("Détection des piétons", detection_piétons),
71  ]
72
73  frame = ttk.Frame(main_window, style="Custom.TFrame")
74  frame.pack(fill='y', side='left', anchor='n')
75
76  for i, (text, command) in enumerate(buttons):
77      button = ttk.Button(frame, text=text, command=command, style="Custom.TButton")
78      button.pack(padx=10, pady=5, fill='x')
79
```

<fonction bouton détection statique>:

```
80     figure = Figure(figsize=(12, 8))
81     axes = [figure.add_subplot(2, 2, i + 1) for i in range(3)]
82     axes[0].set_title("Image")
83     axes[1].set_title("Gradient")
84     axes[2].set_title("Résultat")
85     figure.subplots_adjust(wspace=0.5, hspace=0.5)
86     figure_canvas = FigureCanvasTkAgg(figure, main_window)
87     figure_canvas.get_tk_widget().pack()
88
89     result_text = tk.StringVar()
90     result_text.set("Prêt")
91     result_label = ttk.Label(main_window, textvariable=result_text, style="Custom.TLabel")
92     result_label.pack(fill='x')
93
94     status = tk.StringVar()
95     status.set("")
96
97     status_label = ttk.Label(main_window, textvariable=status, style="Custom.TLabel")
98     status_label.pack(fill='x')
99
```

<fonction bouton détection en live>:

```
100 ~ def start_live_detection():
101     #database
102 ~     if not os.path.exists("database"):
103         os.makedirs("database")
104
105     engine = pyttsx3.init()
106
107 ~     def calculer_distance(hauteur_objet, taille_reelle_objet, distance_focale):
108         distance = (taille_reelle_objet * distance_focale) / hauteur_objet
109         return distance
110     main_window = tk.Toplevel(root)
111     main_window.title("Application de Détection des piétons")
112
113     distance_text = tk.StringVar()
114
115     #initialisation de camera module
116     cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
117
118     # Configuration de l'alarme sonore
119     pygame.mixer.init()
120     alarm_sound = pygame.mixer.Sound("/home/zaraki/Téléchargements/Fire_pager-jason-1283464858.wav")
```

<fonction bouton détection en live>:

```
122  def capture_frame():
123      _, frame = cap.read()
124      frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
125      display_image(frame, 1)
126      detect_pedestrians(frame)
127
128      main_window.after(1, capture_frame)
129
130  def detect_pedestrians(frame):
131      hog = cv2.HOGDescriptor()
132      hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
133      rects, weights = hog.detectMultiScale(frame, winStride=(8, 8))
134
135      false_detections = 0
136      if len(rects) > 0:
137          boxe = []
138          for (x, y, w, h) in rects:
139              boxe.append((x, y, x + w, y + h))
140          for i, (x, y, w, h) in enumerate(rects):
141              if weights[i] < 0:
142                  false_detections += 1
143              cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255), 2)
```

<fonction bouton détection en live>:

```
144 #pourcentage
145 detection_percentage = int((len(rects) - false_detections) / len(rects) * 100)
146 result_text.set(f"Probabilité de détection: {detection_percentage}% - Fausses détections: {false_detections}")
147
148 for (x1, y1, x2, y2) in boxe:
149     hauteur_objet = y2 - y1
150     distance = calculer_distance(1.7, hauteur_objet, 3.04/1000)
151     cv2.putText(frame, f"Distance: {distance:.2f}m", (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 2)
152
153     # Enregistrer l'image du piéton
154     pedestrian_image = frame[y1:y2, x1:x2]
155     image_path = os.path.join("database", f"pedestrian_{len(os.listdir('database'))}.jpg")
156     cv2.imwrite(image_path, pedestrian_image)
157
158     # Alarme sonore
159     if distance < 2.0:
160         alarm_sound.play()
161         engine.say("Danger! Un piéton proche!")
162         engine.runAndWait()
163     elif distance < 5.0:
164         alarm_sound.play()
165         engine.say("Attention! Un piéton à proximité!")
166         engine.runAndWait()
167 distance_text.set(f"Distance: {distance} m")
```

<fonction bouton détection en live>:

```
170 ~ else:
171     result_text.set("Aucune détection")
172     distance_text.set("")
173     # Arrêter le son d'alarme s'il est en cours de lecture
174     alarm_sound.stop()
175     display_image(frame, 2)
176
177 ~ def display_image(img, axis):
178     axes[axis - 1].clear()
179
180     axes[axis - 1].imshow(img, cmap='gray' if len(img.shape) == 2 else None)
181     figure_canvas.draw()
182
183 ~ def stop_detection():
184     cap.release() # Libérer la caméra Raspberry Pi
185     main_window.destroy() # Fermer la fenêtre principale
186
187     buttons = [("Arrêter la détection", stop_detection)]
188     frame = ttk.Frame(main_window, style="Custom.TFrame")
189     frame.pack(fill='y', side='left', anchor='n')
190 ~ for i, (text, command) in enumerate(buttons):
191     button = ttk.Button(frame, text=text, command=command, style="Custom.TButton")
192     button.pack(padx=10, pady=5, fill='x')
```

<fonction bouton détection en live>:

```
194 figure = Figure(figsize=(16, 6))
195 axes = [figure.add_subplot(1, 2, i + 1) for i in range(2)]
196 axes[0].set_title("Image")
197 axes[1].set_title("Résultat")
198 figure.subplots_adjust(wspace=0.2)
199 figure_canvas = FigureCanvasTkAgg(figure, main_window)
200 figure_canvas.get_tk_widget().pack(side="top", fill="both", expand=True)
201
202 result_text = tk.StringVar()
203 result_text.set("Prêt")
204 result_label = ttk.Label(main_window, textvariable=result_text, style="Custom.TLabel")
205 result_label.pack(fill='x')
206
207 capture_frame()
208
```

<3eme bouton>:

```
16 ▾ def show_images():  
17     repertoire_images = "database"  
18     pattern = os.path.join(repertoire_images, "*.jpg")  
19     liste_images = glob.glob(pattern)  
20  
21 ▾ for chem in liste_images:  
22     image = cv2.imread(chem)  
23     cv2.imshow("Image", image)  
24     cv2.waitKey(0)  
25     cv2.destroyAllWindows()  
26
```

```

209 root = tk.Tk()
210 root.title("Bienvenue")
211 root.geometry("817x600")
212
213
214 a = tk.Label(root, text="Bienvenue dans l'application de détection ", font=('Montserrat', 20))
215 a.grid(row=0, column=1)
216
217 start_button = ttk.Button(root, text="Démarrer", command=start_main_window, style="Custom.TButton")
218 start_button.grid(row=3, column=0, padx=10, pady=5)
219
220 exit_button = ttk.Button(root, text="Quitter", command=root.quit, style="Custom.TButton")
221 exit_button.grid(row=3, column=2, padx=10, pady=5)
222
223 live_detection_button = ttk.Button(root, text="Détection en direct", command=start_live_detection, style="Custom.TButton")
224 live_detection_button.grid(row=3, column=1, padx=10, pady=5)
225
226 show_images_button = ttk.Button(root, text="Afficher les images", command=show_images, style="Custom.TButton")
227 show_images_button.grid(row=4, column=1, padx=10, pady=5)
228
229
230 root.mainloop()

```

Distance (m)	Nbr de piétons détectés		précision	
	HOG	HAAR	HOG	HAAR
0	0	0	0%	0%
1	0	0	0%	0%
2	0	0	0%	0%
3	5	5	33,33%	33,33%
4	7	7	46,60%	46,60%
5	12	6	80%	40%
6	15	10	100%	66,66%
7	14	4	93,30%	2,66%
8	0	0	0,00%	0%
9	0	0	0,00%	0%
10	0	0	0,00%	0%

```

1  import matplotlib.pyplot as plt
2
3  x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4  precision_hog = [0, 0, 0, 33.33, 46.60, 80, 100, 93.30, 0, 0, 0]
5  precision_haar = [0, 0, 0, 33.33, 46.60, 40, 66.66, 2.66, 0, 0, 0]
6
7  fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))
8
9  ax1.plot(x, precision_hog, label='HOG')
10 ax1.set_xlabel('Abcisse')
11 ax1.set_ylabel('Précision (%)')
12 ax1.set_title('Précision - HOG')
13 ax1.legend()
14
15 ax2.plot(x, precision_haar, label='HAAR')
16 ax2.set_xlabel('Abcisse')
17 ax2.set_ylabel('Précision (%)')
18 ax2.set_title('Précision - HAAR')
19 ax2.legend()
20
21 plt.tight_layout()
22 plt.show()

```